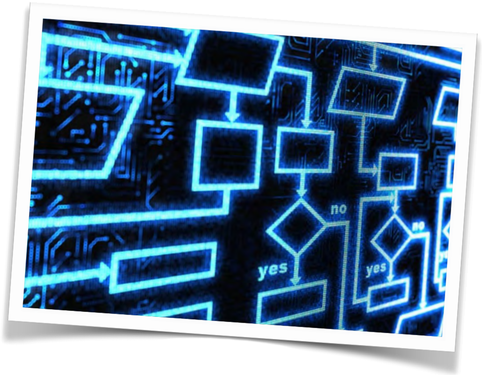
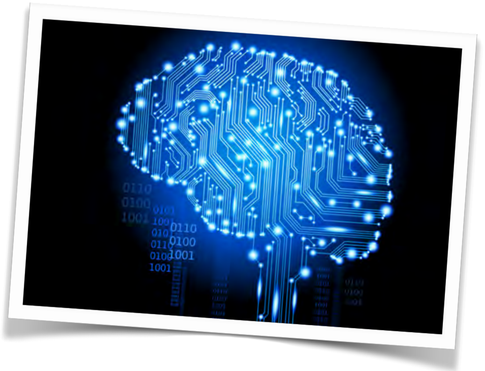
functions



memory

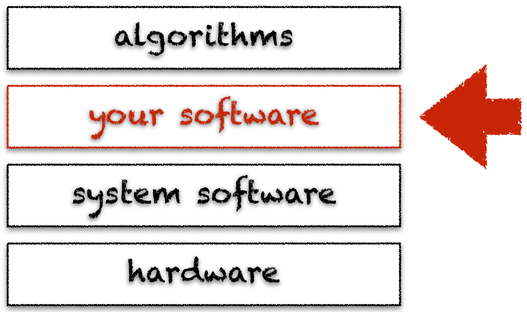


&

errors



learning objectives

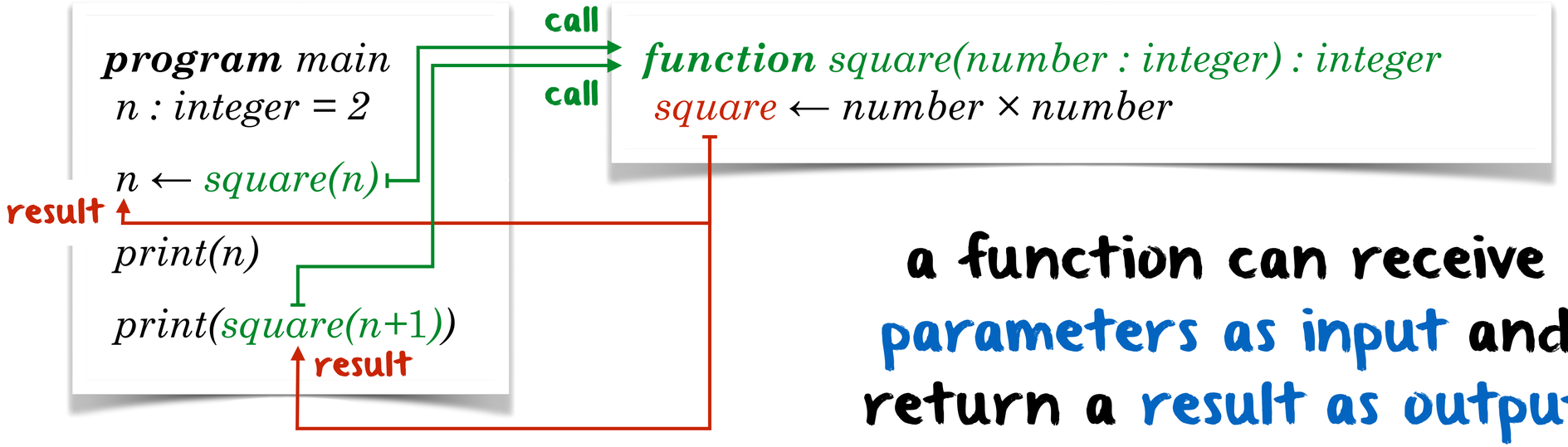


- learn about functions and how to use them
- learn about memory management
- learn about errors and how to handle them

what's a function?

in a program, a function is a symbolic name (identifier) associated with a sequence of instructions that performs a specific task

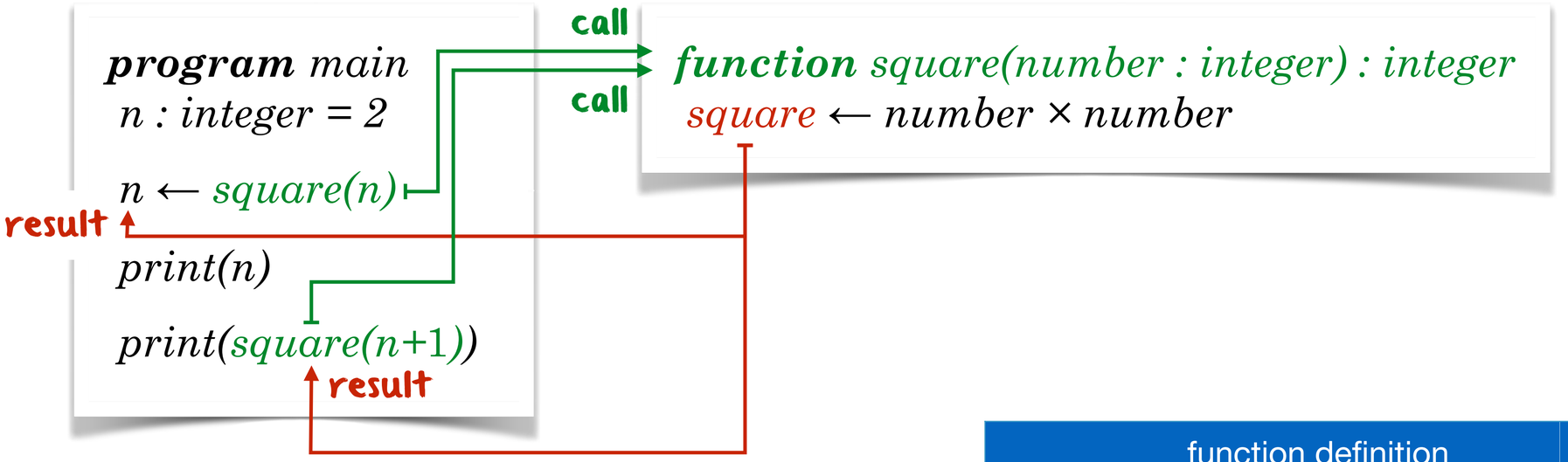
once defined, a function can then be called in programs wherever that particular task should be performed



a function can receive parameters as input and return a result as output

function ⇔ procedure ⇔ routine ⇔ subroutine ⇔ subprogram ⇔ method

what's a function?



	function definition	function call
python	`def square(number):` `  return number * number`	`result = square(2)`

java	`public int square(int number) {`
	`  return number * number`
	`}`
		`int result = square(2)`

```
function isLeap(year : integer)
if  year mod 400 = 0
    isLeap ← true
else if  year mod 100 = 0
    isLeap ← false
else if  year mod 4 = 0
    isLeap ← true
else  isLeap ← false
```







a function to test leap years

```
function isLeap(year : integer)
if  ((year mod 4 = 0)  ∧  (year mod 100 ≠ 0))   ∨   (year mod 400)
    isLeap ← true
else
    isLeap ← false
```

```
function isLeap(year : integer)
isLeap  ←  ((year mod 4 = 0)  ∧  (year mod 100 ≠ 0))   ∨   (year mod 400)
```







a function to test leap years

python



```
def isLeap(year):
    if year % 400 == 0 : return True
    elif year % 100 == 0 : return False
    elif year % 4 == 0 : return True
    return False
```

```
def isLeap(year):
    if (year % 4 == 0) and (year % 100 != 0) or (year % 400 == 0) : return True
    return False
```

```
def isLeap(year):
    return (year % 4 == 0) and (year % 100 != 0) or (year % 400 == 0)
```







a function to test leap years

java



```
public class LeapYear {
  public static boolean isLeap(int year) {
    if (year % 400 == 0) return true;
    if (year % 100 == 0) return false;
    if (year % 4 == 0) return true;
    return false;
  }
}
```

```
public class LeapYear {
  public static boolean isLeap(int year) {
    if ((year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0))
      return true;
    else return false;
}
```

```
public class LeapYear {
  public static boolean isLeap(int year) {
    return (year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0);
  }
}
```

predefined functions



most programming languages come with
a rich library of predefined functions

this is the case of input/output functions, which allow you to ask the user for text input and to provide text output in the terminal

	input	output
python	`year = input("Give us a year: ")` `year = int(year)`	`print("Is {0} a leap year? {1}".format(year, isLeap(year)))`

java	`import java.util.Scanner;`
	
	`Scanner scanner = new Scanner(System.in);`
	`int year = scanner.nextInt();`
		`System.out.println("Is " + year + " a leap year? " + isLeap(year));`

LeapYear

LeapYear.java

LeapYear.py

input/output functions

```
my-computer ~ % cd LeapYear
my-computer LeapYear % ls -l
total 16
-rw-r--r--  1 garbi  staff  114 Sep 10 13:40 LeapYear.java
-rw-r--r--  1 garbi  staff   21 Sep 10 13:41 LeapYear.py
```



java

```
public class LeapYear {
   public static boolean isLeap(int year) { … }
   public static void main(String[] args) {
     System.out.print("Give us a year: ");
     Scanner scanner = new Scanner(System.in);
     int year = scanner.nextInt();
     System.out.println("isLeap(" + year + ") = " + isLeap(year));
   }
}
```

```
my-computer LeapYear % javac LeapYear.java
my-computer LeapYear % java LeapYear
Give us a year: 2000
isLeap(2000) = true
my-computer LeapYear % java LeapYear
Give us a year: 2001
isLeap(2001) = false
```

python



```
def isLeap(year): …

if __name__ == "__main__":
   year = int(input("Give us a year: "))
   print("isLeap({0}) = {1}".format(year,isLeap(year)))
```

```
my-computer LeapYear % python LeapYear.py
Give us a year: 2000
isLeap(2000) = True
my-computer LeapYear % python LeapYear.py
Give us a year: 2001
isLeap(2001) = False
```

LeapYear

LeapYear.java

LeapYear.py

command line arguments

```
my-computer ~ % cd LeapYear
my-computer LeapYear % ls -l
total 16
-rw-r--r--  1 garbi  staff  114 Sep 10 13:40 LeapYear.java
-rw-r--r--  1 garbi  staff   21 Sep 10 13:41 LeapYear.py
```



java

```
public class LeapYear {
   public static boolean isLeap(int year) { … }
   public static void main(String[] args) {
     if (args.length == 0) {
          System.out.println("Pass a year as argument on the command line!");
      } else {
        int year = Integer.parseInt(args[0]);
        System.out.println("isLeap(" + year + ") = " + isLeap(year));
      }
   }
}
```

```
my-computer LeapYear % javac LeapYear
my-computer LeapYear % java LeapYear
Pass a year as argument on the command line!
my-computer LeapYear % java LeapYear 2000
isLeap(2000) = true
my-computer LeapYear % java LeapYear 2001
isLeap(2001) = false
```

python



```
import sys

def isLeap(year): …

if __name__ == "__main__":
  if len(sys.argv) == 1:
     print("Pass a year as argument on the command line!")
  else:
     year = int(sys.argv[1])
     print("isLeap({0}) = {1}".format(year,isLeap(year)))
```

```
my-computer LeapYear % python LeapYear.py
Pass a year as argument on the command line!
my-computer LeapYear % python LeapYear.py 2000
isLeap(2000) = True
my-computer LeapYear % python LeapYear.py 2001
isLeap(2001) = False
```

LeapYear

LeapYear.java

LeapYear.py

command line arguments

alternative example

```
my-computer ~ % cd HelloWorld
my-computer HelloWorld % ls -l
total 16
-rw-r--r--  1 garbi  staff  114 Sep 10 13:40 HelloWorld.java
-rw-r--r--  1 garbi  staff   21 Sep 10 13:41 HelloWorld.py
```



java

```java
public class HelloWorld {
   public static void main(String[] args) {
       if (args.length == 0) {
           System.out.println("Hello World!");
        } else {
           System.out.println("Hello " + args[0] + "!");
        }
   }
}
```

```
my-computer HelloWorld % javac HelloWorld.java
my-computer HelloWorld % java HelloWorld
Hello World!
my-computer HelloWorld % java HelloWorld James
Hello James!
my-computer HelloWorld % java HelloWorld James Gosling
Hello James!
```

python



```python
import sys

if __name__ == "__main__":
  if len(sys.argv) == 1:
     print("Hello World!")
  else:
     print("Hello {0}!".format(sys.argv[1]))
```

```
my-computer HelloWorld % python HelloWorld.py
Hello World!
my-computer HelloWorld % python HelloWorld.py Guido
Hello Guido!
my-computer HelloWorld % python HelloWorld.py Guido van Rossum
Hello Guido!
```

the scope of a variable

each identifier has a scope*, which refers to the part of the program in which the identifier can be used

*sometimes called the lexical scope

variable shadowing* occurs when a variable declared in a scope has the same name as one declared in an outer scope

*sometimes called variable hiding

```python
place = "Earth"

def continent():
    place = "Europe"

    def country():
        place = "Switzerland"
        print(f"your country is {place}")

    country()
    print(f"your continent is {place}")

continent()
print(f"your planet is {place}")
```



python



```
my-computer VariableShadowing % python ShadowingExample.py
your country is Switzerland
your continent is Europe
your planet is Earth
```

memory management

the runtime system is responsible making the various parts of software work together during execution

in particular, it is responsible for allocating and deallocating memory blocks used by the software

memory blocks hold your data structures stored in or accessed through variables

depending on the need, the runtime system fetches such blocks from two distincts memory zones:



heap or stack



the heap



in the heap, allocating is easy

but deallocating can be difficult*

*difficult ⇔ potentially unsafe

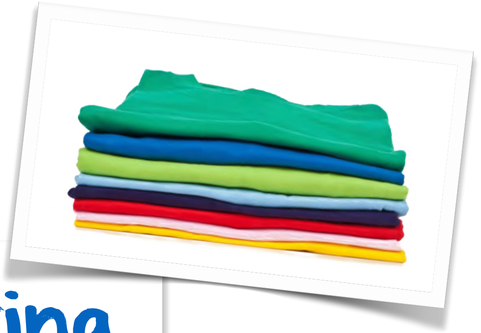
memory blocks can be allocated and deallocated in a any order

this is difficult if you create data structures and pass them to one or more functions

so deallocation implies to be sure that deallocated memory blocks will not be needed in the future

we will come back to this problem when we will introduce object-oriented programming

the stack



in the stack, deallocating is as easy as allocating

because memory blocks are allocated and deallocated in a predefined order known as lifo (last in first out)

this implies that a memory block cannot be deallocated before more recently allocated blocks are deallocated first

this is precisely the case for local variables and parameters allocated in nested method calls

java

the call stack



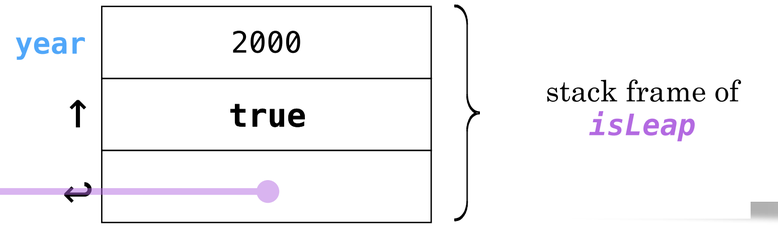
```
public class LeapYear {

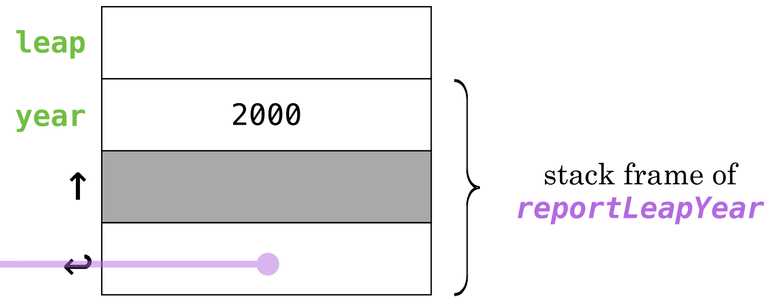
  static boolean isLeap(int year) {
    return (year % 4 == 0) && (year % 100 != 0) || (year % 400 == 0);
  }

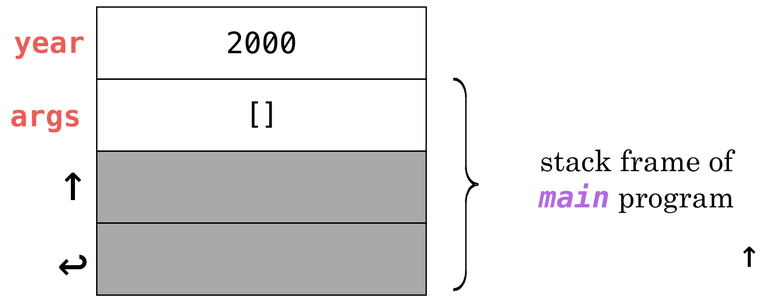
  static void reportLeapYear(int year) {
    boolean leap = isLeap(year);
    System.out.println("isLeap(" + year + ") = " + leap );
  }

  public static void main(String[] args) {
    int year = 2000;
    reportLeapYear(year);
  }
}
```

breakpoint here







↑ *returned value for the caller*

↩ *return address in the caller*

the call stack contains a stack frame for each function call currently active

a stack frame* contains all the local variables and parameters of the function being called

*stack frame ⇔ memory block



managing errors

program errors can have multiple causes

incorrect implementation, e.g., division by 0

specification not met by the implementation

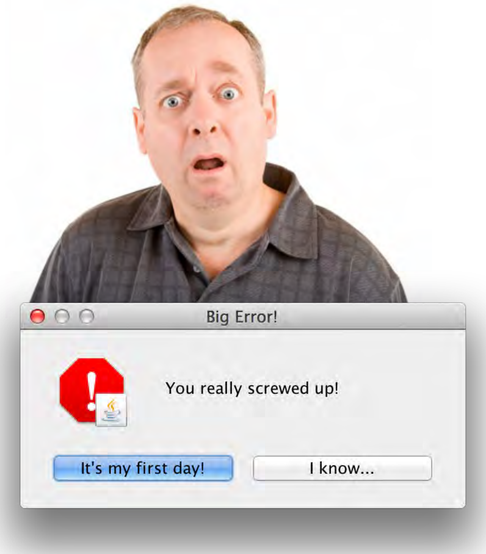
note also that it is not always the programmer's fault

missing files

lack of appropriate permissions

network interruption

incorrect url entered by the user



managing errors

when you define a function, should you assume it will be used properly?

or should you assume it will possibly be used the wrong way?

for functions, arguments represent a major vulnerability

```
balance = 0

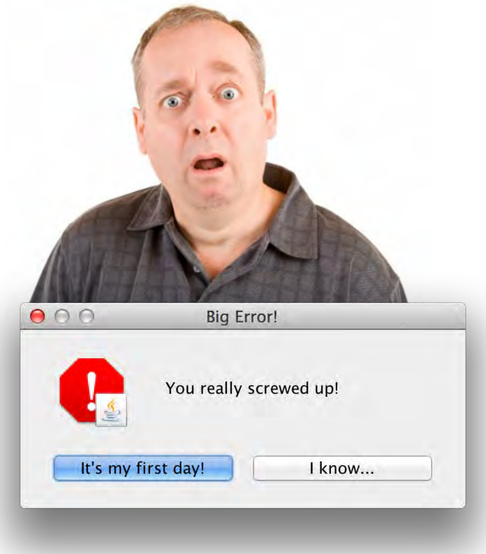
def deposit(amount):
    global balance
    balance = balance + amount

def withdraw(amount):
    global balance
    balance = balance - amount
```

```
deposit(100)
print(f"balance = {balance}")

withdraw(500)
print(f"balance = {balance}")
```

```
my-computer Banking % python Banking.py
balance = 100
balance = -400
```



defensing programming

hope for the best but
prepare for the worse

```
balance = 0

def deposit(amount):
    global balance
    balance = balance + amount

def withdraw(amount):
    global balance
    balance = balance - amount
```

functions

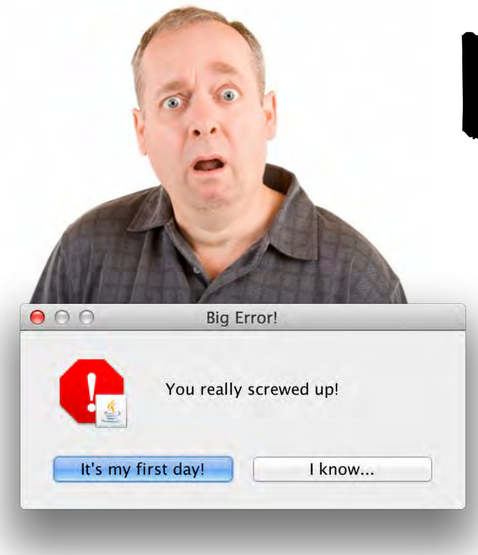
how should functions report errors?

```
deposit(100)
print(f"balance = {balance}")

withdraw(500)
print(f"balance = {balance}")
```

callers

how should callers of functions handle errors?



how to report errors and handle errors

a first solution

return some error code when an error occurs, e.g., as a negative number

report errors

```
balance = 0

def withdraw(amount):
    global balance
    if amount > balance:
        return -1
    else:
        balance = balance - amount
        return 0
```

function

what if the caller ignores the error code?

handle errors

```
amount = 500
if withdraw(amount) < 0:
    print(f"you cannot withdraw {amount} with a balance of {balance}")
else:
    print(f"balance = {balance}")
```

caller



how to report errors and handle errors

a second solution

throw an exception when an error occurs



report errors

function

```
def withdraw(amount):
    global balance
    if amount > balance:
        raise ValueError("insufficient balance")
    else:
        balance = balance - amount
```

print(…) is never called !

if you do not handle exceptions

caller

```
amount = 500
withdraw(amount)
print(f"you cannot withdraw {amount} with a balance of {balance}")
```

```
my-computer Banking % python Banking.py
Traceback (most recent call last):
  File "/Users/garbi/Banking/Banking.py", line 12, in <module>
    withdraw(amount)
  File "/Users/garbi/Banking/Banking.py", line 3, in withdraw
    raise ValueError("insufficient balance")
ValueError: insufficient balance
```



how to report errors and handle errors

a second solution

how to handle exceptions

catch the exception when it occurs



caller

```
try:
    withdraw(amount)
    print(f"balance = {balance}")
except ValueError as err:
    print(f"unable to withdraw {amount} because of {err}")
```

```
my-computer Banking % python Banking.py
unable to withdraw 300 because of insufficient balance
```

caller

```
try:
    withdraw(amount)
except ValueError as err:
    print(f"unable to withdraw {amount} because of {err}")
finally:
    print(f"balance = {balance}")
```

```
my-computer Banking % python Banking.py
unable to withdraw 300 because of insufficient balance
balance = 100
```

caller

```
try:
    amount = amount / 0
    withdraw(amount)
except ValueError as err:
    print(f"unable to withdraw {amount} because of {err}")
except ZeroDivisionError as err:
    print(f"exception raised: {err}")
```

```
my-computer Banking % python Banking.py
exception raised: division by zero
```



how to report errors and handle errors

a second solution

throw an exception
when an error occurs



report errors

function

```
void withdraw(double amount) {
  if (amount > balance) {
     throw new IllegalArgumentException("insufficient balance");
  }
  balance = balance - amount;
}
```

print(…) is never called !

in java this time!

if you do not handle exceptions

caller

```
var amount = 500;
withdraw(amount);
System.out.println("balance = " + balance);
```

```
my-computer Banking % java Banking
Exception in thread "main"
java.lang.IllegalArgumentException: insufficient balance
    at ch.doplab.Banking.withdraw(Banking.java:20)
    at ch.doplab.Banking.main(Banking.java:30)
```



how to report errors and handle errors

a second solution

how to handle exceptions

catch the exception when it occurs



caller

```
try {
    withdraw(amount);
} catch (IllegalArgumentException ex) {
    System.out.println("unable to witdraw " + amount + " because of " + ex.getMessage());
} finally {
    System.out.println("balance = " + balance);
}
```

```
my-computer Banking % java Banking
unable to witdraw 500 because of insufficient balance
balance = 100.0
```

caller

```
try {
    amount = amount / 0;
    withdraw(amount);
} catch (IllegalArgumentException ex) {
    System.out.println("unable to witdraw " + amount + " because of " + ex.getMessage());
} catch (ArithmeticException ex) {
    System.out.println("exception raised: " + ex.getMessage());
} finally {
  System.out.println("balance = " + balance);
}
```



```
my-computer Banking % java Banking
exception raised: / by zero
balance = 100.0
```



how to report errors and handle errors

a second solution

in java, the compiler can force the function to declare the exceptions it throws and the caller to handle them



function

```
void withdraw(double amount) {
  if (amount > balance) {
     throw new InsufficientBalanceException();
  }
  balance = balance - amount;
}
```

```
my-computer Banking % javac Banking.java
/Users/garbi/Banking/Banking.java:20: error: unreported exception
InsufficientBalanceException; must be caught or declared to be thrown
```

```
void withdraw(double amount) throws InsufficientBalanceException {
  if (amount > balance) {
     throw new InsufficientBalanceException();
  }
  balance = balance - amount;
}
```

exception declaration

why is there still a compilation error?

caller

```
try {
   withdraw(amount);
} catch (IllegalArgumentException ex) {
    System.out.println("unable to witdraw " + amount + " because of " + ex.getMessage());
}
```

```
my-computer Banking % javac Banking.java
/Users/garbi/Banking/Banking.java:31: error: unreported exception
InsufficientBalanceException; must be caught or declared to be thrown
```



how to report errors and handle errors

a second solution

in java, the compiler can force the function to declare the exceptions it throws and the caller to handle them



function

```
void withdraw(double amount) throws InsufficientBalanceException {
  if (amount > balance) {
     throw new InsufficientBalanceException();
  }
  balance = balance - amount;
}
```

exception declaration

IllegalArgumentException ≠ InsufficientBalanceException

why is there still a compilation error?

caller

```
try {
   withdraw(amount);
} catch (IllegalArgumentException ex) {
    System.out.println("unable to witdraw " + amount + " because of " + ex.getMessage());
}
```



how to report errors and handle errors

a second solution

in java, the compiler can force the function to declare the exceptions it throws and the caller to handle them



function

```
void withdraw(double amount) throws InsufficientBalanceException {
  if (amount > balance) {
     throw new InsufficientBalanceException();
  }
  balance = balance - amount;
}
```

exception declaration

now it compiles and executes properly

caller

```
try {
   withdraw(amount);
} catch (InsufficientBalanceException ex) {
    System.out.println("unable to witdraw " + amount + " because of " + ex.getMessage());
}
```

why does the compiler require the function to declare InsufficientBalanceException but not IllegalArgumentException



checked vs. unchecked exceptions in java

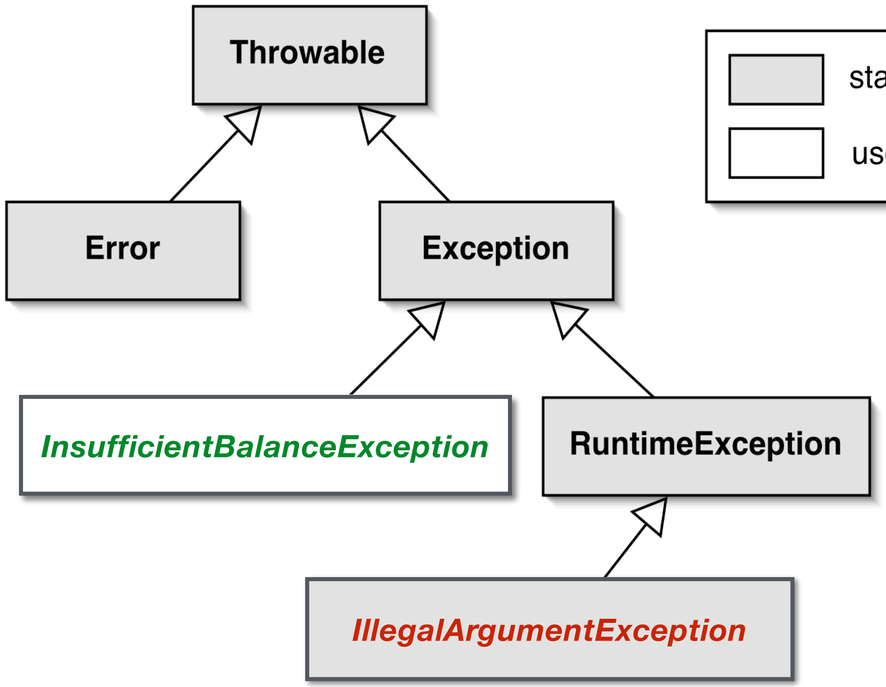


why does the compiler require the function to declare `InsufficientBalanceException` but not `IllegalArgumentException`



checked

for anticipated failures where recovery may be possible





unchecked

for unanticipated failures where recovery is very unlikely